工場リフト呼出システム:アーキテクチャ提案および概算見積

文書	アーキテクチャ提案および概算見積(概要版)
元資料	「工場リフト呼出システム 提案書」(第1版・第2版)および画面モック
ステータス	レビュー用ドラフト
日付	2026-10-28

1. 概要

5台の生産設備(号機)の作業者が、**製品引取・戻材引取・トラブル**の理由でリフト担当者を簡単に呼び出せる仕組みが必要です。リフト担当者は呼出を順番どおりに確認して応答し、双方の画面でリフトが運転中かどうかが分かる必要があります。

当初の提案書では、タブレット上で**2種類のWPFネイティブアプリ**を動かす構成でした。本書では、よりシンプルな構成として、構内の小型サーバーから配信する**1つのWebアプリケーション**を、**Windowsキオスクモードの Microsoft Edge** で表示する方式を推奨します。お客様が選定された技術(**.NET、SQL Server、IIS**)の範囲内に収まり、既存の SQL Server ライセンスを活用します。主な変更点は次の3つです。

- **タブレットへのインストール・更新が不要です。** 新バージョンはサーバーに1回配置するだけです。
- **構成要素が少なくなります。** コードベースも配置物も1つで、画面はすでにWebページとしてデザインされています。
- **Windowsタブレットに縛られません。** 将来 Android タブレットや壁掛けモニターを使う場合も、同じURLを開くだけです。

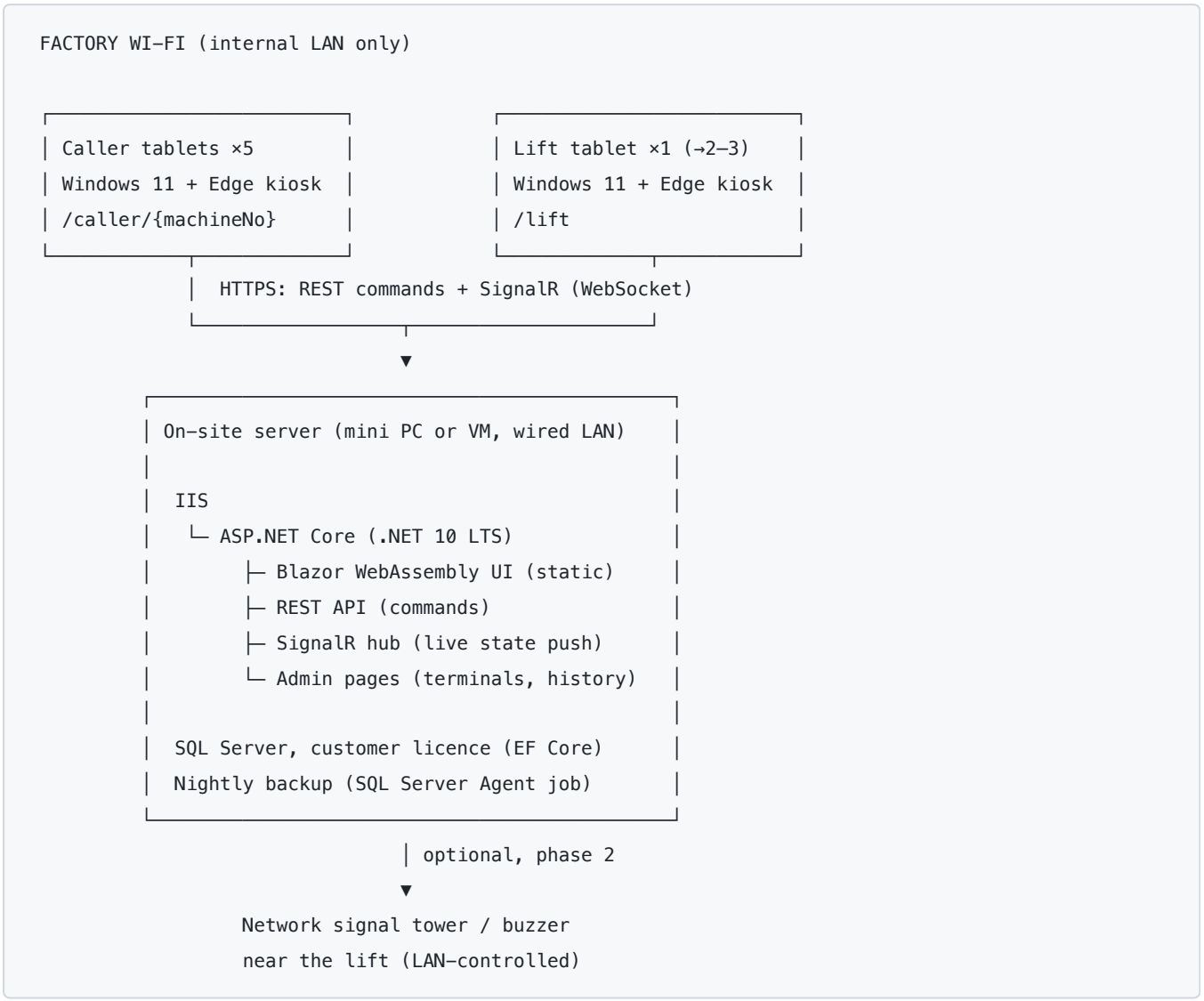
概算: 開発者1名とパートタイムのテスター1名で、現場での2週間の試験運用を含め、**約12週間**で本番稼働できます。AIを活用した開発では**約8〜9週間**です。

2. 要件の整理

項目	要件
呼出元	タブレット5台(1〜5号機)。タッチ操作のみ。呼出ボタン3種と呼出解除ボタン
呼出元の表示	呼出に応答中かどうか、リフトが運転中か停止中か
リフト側	当面タブレット1台(将来2〜3台)。どの号機から、どの理由で、どの順番で呼ばれたかを表示
リフト側の操作	呼出への応答、リフトの運転/停止の切替
ネットワーク	全端末が同一の工場Wi-Fiに接続。社内LANのみ
外部連携	PLC連携なし。リフトの状態は担当者が手動で切替
技術スタック	全体を **.NET / C#** で統一、**SQL Server**(お客様保有ライセンス)、IIS。当初のクライアント案はWPF

3. 推奨アーキテクチャ

3.1 全体構成



凡例: Caller tablets = 呼出元タブレット、Lift tablet = リフト側タブレット、Edge kiosk = Edgeキオスクモード、On-site server = 構内サーバー(ミニPCまたはVM、有線LAN)、REST commands = RESTコマンド、live state push = 状態のリアルタイム配信、Admin pages = 管理画面(端末・履歴)、customer licence = お客様ライセンス、Nightly backup = 夜間バックアップ、optional, phase 2 = オプション(フェーズ2)、Network signal tower / buzzer near the lift = リフト付近のLAN制御信号灯・ブザー

3.2 呼出の流れ

1. 3号機の作業者が**製品引取**をタップします。タブレットは自ら生成したリクエストIDを付けて `POST /api/calls` を送信するため、再送しても呼出が重複することはありません。
2. サーバーは呼出を1つのトランザクションで保存し、新しい**状態スナップショット**(すべての有効な呼出とリフト状態)を SignalR で接続中の全画面へプッシュします。
3. リフト側タブレットは音を鳴らし、呼出をキューの末尾に表示し、誰かが応答するまでアラートを繰り返します。
4. リフト担当者が**応答**をタップします。サーバーは呼出がまだ「呼出中」の場合のみ受け付けるため、リフト側端末が2〜3台あっても最初のタップだけが有効になります。新しいスナップショットにより、3号機の画面が緑色の「応答中」に変わります。

5. 引取が終わると担当者が**完了**をタップし、呼出は履歴に移ります。

3.3 主な設計判断

判断項目	選択	理由
クライアント方式	WPFではなく**Edgeキオスクモード上のWebアプリ**	更新はサーバー側のみで済みます。Windows標準のキオスク機能(割り当てられたアクセス)があり、ブラウザがクラッシュしても自動で再起動します。将来ほかの端末でも使えます
UIフレームワーク	**Blazor WebAssembly**(C#)	最初から最後までC#で開発できます。UIはブラウザ内で動くため、Wi-Fiが切れても画面が失われず、独自のエラー表示ができます。その他の.NETクライアント方式は§5.3で比較しています
リアルタイム更新	**SignalR(WebSocket)**で変更のたびに**全体スナップショット**を送信し、**ポーリングを自動フォールバック**として併用	状態は小さい(多くても数十件の呼出)ため、差分ではなく全体を送る方がシンプルで確実です。再接続後も、次のスナップショットが届いた時点で画面が正しい状態になります。その他の方式(Webhook、メッセージキュー、MQTT)は§5.2で比較しています
コマンド	再送しても安全な**リクエストID付きREST**(冪等性)	Wi-Fiの瞬断後に再送しても2件目の呼出は作られません。RESTはテストやログ記録も容易です
同時実行制御	条件付き更新(`WHERE Status = 'Calling'`)と行バージョン	リフト側端末が複数あっても、同じ呼出に2人が応答することはありません
データベース	お客様保有ライセンスの**SQL Server**をEF Core経由で利用	追加ライセンス費用が不要で、お客様のIT部門が運用・バックアップに慣れています。ライセンス条件に応じて、ネットワーク上の既存インスタンスでも、アプリサーバーへの新規インストールでも構いません。バックアップと保守は SQL Server エージェントで行います。データ量は1日数百行程度です
ホスティング	**IIS**、ASP.NET Core インプロセスホスティング	お客様のご要望どおりです。WebSocketの有効化と、長時間接続向けのアプリケーションプール設定が必要です(§6参照)
ランタイム	**.NET 10 LTS**	当初提案の .NET 8 は**2026年11月**にサポートが終了します。.NET 10 は2028年11月までサポートされます
端末の識別	各タブレットに**端末トークン**付きの専用起動URLを設定(例: `/caller/3?t=…`)	現場でのログイン画面は不要ですが、すべての要求を特定のタブレットまで追跡できます。トークンは管理画面で無効化できます
通信断の検知	**SignalR接続そのものから接続状態を判定**し、クライアントのハートビートで補完	別の監視システムなしで、リフト側画面に「4号機 通信断」を表示できます

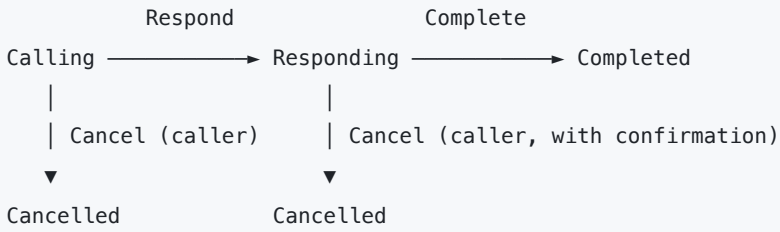
3.4 技術スタック

レイヤー	技術
タブレットOS/シェル	Windows 11 Pro、割り当てられたアクセスで **Microsoft Edge キオスクモード** を実行
フロントエンド	Blazor WebAssembly(.NET 10)、既存モックに基づくCSS、日本語・英語のリソースファイル
リアルタイム	ASP.NET Core SignalR(WebSocket、自動フォールバックあり)
バックエンド	ASP.NET Core 10 の Web API と SignalR ハブ、IISでホスト
データ	SQL Server(お客様ライセンス、2019以降)、EF Core 10(マイグレーション)
ログ	Serilog(ローリングファイル)とデータベースの監査テーブル
死活監視	ASP.NET Core ヘルスチェック(`/health`)

レイヤー	技術
テスト	xUnit、`WebApplicationFactory` による結合テスト、UIコンポーネント用の bUnit、ソークテスト用の簡易端末シミュレーター
CI/CD	GitHub Actions または Azure DevOps:ビルド、テスト、zip発行。IISへの配置はスクリプトによるコピー(サーバーにインターネット接続は不要)
バックアップ	SQL Server エージェントのジョブでネットワーク共有へ毎晩フルバックアップ、またはお客様既存のSQLバックアップ運用

4. 機能設計

4.1 呼出のライフサイクル



凡例: Calling = 呼出中、Responding = 応答中、Completed = 完了、Cancelled = 解除、Respond = 応答、Complete = 完了操作、Cancel (caller) = 呼出元による呼出解除、with confirmation = 確認ダイアログあり

4.2 画面

画面	内容
呼出元(`/caller/{n}`)	号機番号、リフト状態(運転中/停止中)、応答状態(待機/呼出中/応答中)、大型の呼出ボタン3つ、呼出解除(呼出中のみ有効)、通信断時の全画面「通信エラー」表示
リフト側(`/lift`)	運転/停止の切替(長押しと確認)、呼出キュー(順番、号機、色・アイコン・文字による理由表示、経過時間、応答ボタン)、完了ボタン付きの「対応中」欄、端末の接続状態表示。新規呼出時に音で通知し、応答があるまでN秒ごとに繰り返し、3分経過でカードを点滅
管理(`/admin`、事務所PCのみ)	端末一覧(接続状態、最終通信時刻、トークン再発行)、絞り込みとCSV出力付きの呼出履歴、設定(再通知間隔、超過判定時間、トラブル優先のON/OFF)

4.3 データモデル(最小構成)

テーブル	主な列
`Terminal`	Id、Name、Role(呼出元/リフト側)、MachineNo、TokenHash、LastSeenAt、IsActive
`Call`	Id、MachineNo、CallType、Status、CreatedAt、RespondedAt、RespondedByTerminalId、CompletedAt、CancelledAt、RequestId(一意)、RowVersion
`LiftState`	1行のみ:IsRunning、UpdatedAt、UpdatedByTerminalId
`Setting`	Key、Value
`AuditEvent`	Id、At、TerminalId、Action、CallId、Details

4.4 API一覧

メソッド	エンドポイント	用途
`POST`	`/api/calls`	呼出登録(安全に再送できるようリクエストID付き)
`POST`	`/api/calls/{id}/respond`	応答(リフト側)
`POST`	`/api/calls/{id}/complete`	完了(リフト側)
`POST`	`/api/calls/{id}/cancel`	呼出解除(呼出元の号機のみ)
`PUT`	`/api/lift/state`	運転/停止の設定
`GET`	`/api/state`	全体スナップショット(起動時・再接続時に使用)
`GET`	`/api/history`	絞り込み・CSV出力付きの履歴(管理)
ハブ	`/hubs/state` → `StateChanged(snapshot)`	全画面へのリアルタイム配信

4.5 未確定事項の推奨デフォルト

以下は開発を止めないための推奨デフォルトです。いずれもお客様の確認が必要で、多くは設定で変更できます。

未確定事項	推奨デフォルト
トラブル呼出を優先表示するか	**する。** 赤色で最上段に表示します。設定でOFFにできます
「完了」操作は必要か	**必要。** 応答時間・対応時間のデータが取れるためです。設定した時間を過ぎても完了されない呼出は自動で完了します
1号機あたりの同時呼出数	**呼出種別ごとに1件。** 製品引取の依頼とトラブル報告を同時に行えます
応答のない呼出	30秒ごとに音を繰り返し、3分でカードを点滅させ、記録を残します。エスカレーション(信号灯や監督者への通知)はフェーズ2のオプションです
リフト停止中の呼出	**可能。** 呼出元には赤い「リフト停止中」ヘッダーが表示され、待ち時間が長くなる可能性が分かります
運転/停止の切替担当	リフト担当者がリフト側タブレットで切り替えます
帳票	履歴のCSV出力を含みます。KPIダッシュボードはフェーズ2のオプションです

5. 検討した代替案

5.1 当初提案からの変更点

項目	当初提案	本提案	理由
クライアント	WPFアプリ2本(または起動設定で切り替える1本)	Edgeキオスクモード上のWebアプリ1本	クライアントのインストール・更新が不要。キオスクとクラッシュ復旧が標準機能
ランタイム	.NET 8 LTS	.NET 10 LTS	.NET 8 のサポートは2026年11月に終了
データベース	SQL Server Express を提案	お客様保有ライセンスの SQL Server	費用負担済み。SQL Server エージェントでバックアップ・保守が可能
リアルタイム更新	SignalRのイベントとポーリングへのフォールバック	SignalRで全体スナップショット、再接続時は `GET /api/state`	よりシンプルで、自動的に正しい状態に戻る

項目	当初提案	本提案	理由
重複防止	記載なし	全コマンドにリクエストID	Wi-Fiの再送で呼出が重複しない
管理機能	対象外	簡易管理画面(端末、履歴、CSV出力、設定)	日々の運用に必要
通知	音と画面表示	繰り返し音とオプションの信号灯	リフト運転中の担当者はタブレットを見られない

WPFが必須条件の場合も、サーバー側の設計はそのまま使えます。変わるのはクライアントだけです。追加工数は§8.2を参照してください。

5.2 タブレットへの更新通知方式

本システムは、同一LAN上のタブレット6〜8台、1日数百件の呼出という規模です。新しい呼出は1〜2秒以内に表示されれば十分です。工数の差は§8の見積との比較です。

方式	本システムへの適合性	開発工数	保守	評価
SignalR	ブラウザへのプッシュ、自動再接続、Blazor用のC#クライアント、接続自体から端末の接続状態が分かる	基準	低。IISのWebSocketとアプリケーションプールの設定は初回のみ	**推奨**
ポーリング(1〜2秒ごとに `GET /api/state`)	8台×毎秒1リクエストはサーバーにとって無視できる負荷。各ポーリングがハートビートを兼ねる	やや少ない	最小。接続状態を持たないため、IISのリサイクルやWi-Fi断の影響を受けない	**組み込みのフォールバックとして採用**
Server-Sent Events	プレーンなHTTPによるサーバーからの一方向プッシュ。.NET 10で直接サポート。「RESTコマンドと状態プッシュ」の設計に合う	ほぼ同じ	低	採用可能だが、C#チームにとってSignalRより優れる点はない
WebSocketの直接実装	再接続、ハートビート、メッセージ処理を自前で実装する必要がある	+0.5〜1週	中	割に合わない。SignalRは実績のあるWebSocketのラッパー
Webhook	サーバー間のHTTPコールバック。ブラウザやタブレットは外部からのHTTP呼出を受けられない	対象外	対象外	画面更新には**使えない**。将来、外部への通知(Teams、メール)には利用可能
メッセージキュー(RabbitMQ、Azure Service Bus、Kafka)	多数のバックエンドサービスの疎結合化や大量メッセージのバッファリング向け。ブラウザは直接接続できないため、前段にSignalRが別途必要	+1.5〜2週	高。構築・セキュリティ・監視・バックアップの対象となるサーバーが増える	サーバー1台・端末8台には**過剰**
MQTT(Mosquittoブローカー、.NET用のMQTTnetライブラリなど)	工場(IIoT)で広く使われる軽量なPub/Sub。ブラウザはWebSocket経由で接続。物理ボタン、信号灯、PLCゲートウェイが標準対応していることが多い	+1〜1.5週	中。ブローカー、アクセス制御、TLSの保守が必要	ハードウェア機器を多数追加する場合の**将来の選択肢**

本システムの難しい部分は、呼出順、1件の呼出に対する応答は1回のみ、呼出の重複防止といった業務ルールです。これらはサーバー側の**データベーストランザクション**で担保しています。キューやブローカーはメッセージを運ぶだけでこれらのルールは担保しないため、インフラが増える一方で作業は減りません。

サーバーは常に**全体の状態**を送るため、プッシュ通知の役割は「何かが変わった」と伝えることだけです。ポーリングへの切替やフォールバックは、タイマーで `GET /api/state` を呼ぶだけで済みます。SignalRに接続できない場合はクライアントが自動でこれを行い、その工数は見積に含まれています。

5.3 .NETのクライアント方式

お客様は.NETの継続を希望されているため、.NETのクライアント技術のみを比較します。サーバー側(ASP.NET Core、SignalR、IIS、SQL Server)は3方式とも共通です。

方式	開発工数	保守	適するケース
Blazor WebAssembly(推奨)	基準	低。更新はサーバーへの配置のみ	標準。Wi-Fiが切れてもUIはブラウザ内で動き続け、独自のエラー表示ができる
Blazor Server(UIをサーバー側で実行)	約1〜1.5週少ない。API層が不要で、リアルタイム更新が組み込み	低	Wi-Fi調査で安定した通信が確認できた場合のみ。UIは常時接続に依存し、切断時は「再接続中…」で画面が止まり、長時間の切断ではセッションが失われる。.NET 10で改善されたが、工場Wi-Fiではリスクが残る
WPF(当初提案のWindowsネイティブアプリ)	+2〜3週(§8.2参照)	高め。全タブレットにインストーラーと自動更新の仕組みが必要	ネイティブアプリが必須の場合(ローカル機器の利用、既存WPFアプリとの統一など)

5.4 結論

1. ASP.NET Core 上の **Blazor WebAssembly**、お客様ライセンスの **SQL Server**、IISでのホスティングとします。
2. リアルタイム更新は **SignalR を採用**し、ポーリングを自動フォールバックとします。
3. コアシステムには **Webhook やメッセージキューを使いません**。Webhookは、Teamsやメール通知のためのフェーズ2の小規模な追加機能として利用できます。
4. 物理的な呼出ボタン、信号灯、PLCデータを将来追加する場合は **MQTT を検討**します。
5. **Blazor Server** は開発を約1週間短縮できますが、Wi-Fi調査で安定した通信が確認できた場合に限ります。

6. 配置と運用

6.1 ハードウェア

品目	数量	備考
サーバー	1	小型の産業用ミニPC、または既存サーバー上のVM。Windows Server または Windows 11 Pro、**有線LAN**、固定IPまたはDNS名、**UPS**に接続。データベースはローカルにインストールせず、お客様ネットワーク上の既存SQL Serverインスタンスを使うことも可能
呼出元タブレット	5(+予備1)	Windows 11 Pro(Homeには「割り当てられたアクセス」がない)。手袋で操作できるタッチ画面。常時給電の壁掛けまたはアームマウント
リフト側タブレット	1(+予備1)	同上。工場内ではタブレットのスピーカーでは音量が足りないため、**電源付きの外付けスピーカー**を追加
Wi-Fi	—	全タブレットの設置場所とリフトの移動経路での**電波調査**。これらの端末用にSSIDを1つ用意
信号灯(オプション)	1	リフト付近に設置する、LAN制御のブザー付き積層信号灯(フェーズ2)

6.2 サーバー設定チェックリスト

- IISの **WebSocket プロトコル** 機能をインストールします。これがないと、SignalRは低速な通信方式にフォールバックします。
- アプリケーションプール設定:開始モード **AlwaysRunning**、アイドルタイムアウト **0**、定期リサイクルは無効にするか夜間の定時リサイクルに変更します。
- サイトのプリロードを有効にし、IISの起動時にアプリも起動させます。
- SQL Server:SQL Server エージェントのジョブ(またはお客様既存のバックアップ運用)で毎晩フルバックアップを取得し、ネットワーク共有にコピーして14日間保管します。呼出履歴は1〜3年保管します(削除ジョブ)。
- 社内CAで発行できる場合はHTTPS証明書を使用します。隔離されたLANではHTTPでも許容範囲ですが、HTTPSを推奨します。

6.3 タブレット(キオスク)設定チェックリスト

- Microsoft Edge を実行する**割り当てられたアクセス**(単一アプリのキオスク)を設定し、起動URLはタブレットごとに指定します。閉じたりクラッシュしたりしても、Edgeが自動でページを開き直します。
- Edgeのポリシー:
- アプリのURLで**自動再生を許可**し、タップしなくてもアラート音が鳴るようにします。
- 翻訳のポップアップと初回起動時の案内を無効にします。
- 電源設定:画面を消さず、スリープしないようにします。Wi-Fiアダプターの省電力機能は無効にします。
- 常時給電による電池の膨張を防ぐため、メーカーの**充電上限設定**(例:80%)を使用します。
- Windows Update のアクティブ時間を稼働時間外に設定します。
- 予備タブレットを30分以内にセットアップできる**キッティング手順書**を用意します。

6.4 障害時の動作

障害	利用者に見える状態	復旧
タブレットのWi-Fi断	全画面の「通信エラー」。呼出済みと誤解しないようボタンを無効化	自動で再接続・再同期
サーバーまたはIISの再起動	数秒間、全画面に「通信エラー」	自動で再接続。有効な呼出はDBから復元
サーバーの長時間停止	全画面に「通信エラー」	取り決めた代替手段(電話・無線)を使用。復旧手順書と予備サーバーまたはVMイメージ
ブラウザのクラッシュ	—	キオスクモードがEdgeを自動で再起動

7. 開発計画

2週間スプリントのアジャイル開発とし、各スプリントの終了時に現場でレビューを行います。

週	作業内容
1–2	要件ワークショップ、未確定事項の決定、クリック可能なモックの現場レビュー、技術設計
3–4	スプリント1:プロジェクト構築、CI、データモデル、呼出の状態遷移、REST API、SignalRスナップショット、呼出元画面
5–6	スプリント2:リフト側画面(キュー、タイマー、音声通知、応答・完了、運転/停止の切替)、通信断の検知
7	スプリント3(短期):管理画面、履歴とCSV出力、設定、日本語・英語のUI文言
8	サーバー・キオスクの設定スクリプト、システムテスト1回目

週	作業内容
9	実機タブレットとWi-Fiでの現場テスト、修正、回帰テスト2回目
10–11	UATと**現場での試験運用**(代替手順を用意した上での実運用)、調整
12	本番稼働、運用・管理手順書の引き渡し

テスターは3週目から約50%で参加し(テスト計画)、8〜11週目はほぼフルタイムで作業します。

8. 概算見積

数値は**週単位**の概算です。ASP.NET Core の経験がある中堅〜上級のC#開発者と、3週目からパートタイムで参加するテスターを前提としています。

8.1 工数とスケジュール

フェーズ	週	開発者(人週)	テスター(人週)
要件定義・設計	1–2	1.5	0.5
開発(3スプリント)	3–7	5.5	1.5
配置・システムテスト	8–9	2	2
UATと現場での試験運用	10–11	2	1
本番稼働・引き渡し	12	1	–
合計	**約12週間**	**約12**(幅 9〜17)	**約5**(幅 3〜6)

- **体制:** 開発者1名とテスター1名(前半はパートタイム、8〜11週目はほぼフルタイム)。開発者2名の場合は、本番稼働まで**約8〜9週間**です。
- **含まないもの:** プロジェクト管理・お客様との調整(10〜15%を加算)、ハードウェアの購入・設置、Wi-Fi電波調査、出張費、Windowsライセンス。SQL Serverのライセンスはお客様にご用意いただきます。

8.2 バリエーションとオプション

項目	開発者	テスター	備考
WebではなくWPFクライアント(サーバーは共通)	+2〜3週	+0.5〜1週	XAMLのUI 2本、パッケージングと自動更新、キオスク設定、インストール・更新のテスト
リフト付近の信号灯・ブザー	約1週	約0.5週	新規呼出時に点灯・鳴動。担当者がタブレットから離れている場合に特に有効
KPIダッシュボード(応答時間、号機・種別・時間帯別の呼出数)	1〜1.5週	約0.5週	既存の履歴データを利用
リフトの運転/停止を自動取得するPLC連携	1〜2週	0.5〜1週	PLCの機種と通信方式に依存。事前に技術検証が必要
応答のないトラブル呼出の監督者へのエスカレーション(メール・Teams)	約0.5週	0.5週未満	メールサーバーまたはインターネット接続が必要

8.3 AI活用時の見積

開発者がAIコーディングアシスタント(例:Claude Code)をコード、自動テスト、スクリプト、ドキュメントの大部分に活用し、そのすべてを開発者自身がレビューする場合、工数は次のように減ります。人・ハードウェア・現場に関

わる作業(ワークショップ、現場テスト、UAT、試験運用)はほとんど変わりません。

	通常	AI活用
開発者	約12人週	約7人週(幅 5〜9)
テスター	約5人週	約4人週(幅 3〜5)
本番稼働までの期間	**約12週間**	**約8〜9週間**

- **品質:** E2Eブラウザテストを含む充実した自動テストをAIで低コストに作成できるため、回帰テストのカバー範囲が広くなります。
- **前提条件:** 開発者が.NETに精通し、生成されたコードをすべてレビューすることが前提です。経験の浅い開発者の場合は短縮効果が小さくなり、見つけにくい不具合のリスクが高まります。
- **確度:** AI活用時の数値は経験に基づく判断です。最初のスプリント後に実績を測定し、必要に応じて計画を見直してください。工場のWi-Fiや実機は超過が起きやすい部分のため、予備の工数を確保してください。

9. リスクと対策

リスク	影響	対策
一部の号機の設置場所やリフト経路でWi-Fiが弱い	呼出の見落とし・遅延	本番稼働前の電波調査。障害が分かるよう通信エラーを表示。現場テストで電波を確認
リフト運転中の担当者がタブレットを見られない・音が聞こえない	呼出の待ち時間が長くなる	外付けスピーカー、繰り返し通知、オプションの信号灯(§8.2)
ブラウザがアラート音をブロックする	通知が鳴らない	アプリURLに対するEdgeの自動再生ポリシー。キオスク設定時に確認
IISがアプリをリサイクルする、またはWebSocketが切断される	短時間の切断	§6.2のアプリケーションプール設定。クライアントは自動で再接続・再同期
サーバー1台の故障	システム停止	UPS、夜間バックアップ、復旧手順書、予備VMイメージ、取り決めた電話・無線による代替手段
常時給電のタブレット:電池の膨張、スリープ、Windows Update	端末の停止	充電上限、電源ポリシー、アクティブ時間の設定、予備タブレット
未確定の業務ルールが期限までに決まらない	手戻り	§4.5のデフォルトを設定変更できる形で実装。1〜2週目に決定
WPFの継続が必須条件になる	追加工数	サーバー設計は変更なし。見積は§8.2

10. 前提条件

- お客様に、タブレット、サーバーまたはVM、Wi-Fiネットワーク、ネットワークへのアクセス、レビューとUATのための現場の担当窓口をご用意いただきます。
- 呼出元端末5台、リフト側端末最大3台、1拠点、リフト1基を対象とします。
- UIは日本語で、英語表示にも対応します。ドキュメントは英語で作成します(日本語訳は含みません)。
- 基本範囲には外部システム連携(ERP、MES、PLC)を含みません。
- 開発チームが現場テストのために現地へ入れることを前提とします(9〜11週目)。